

OE-EC804A

Artificial Intelligence

Combined Module Notes

Modules 1 – 6

Semester VIII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: Overview and Foundations of AI	4
1 Overview and Foundations of AI	4
1.1 The Four Definitions of AI	4
1.2 Foundations of AI	4
1.3 History & Evolution of AI	5
1.4 The Turing Test	5
2 Intelligent Agents and Environments	5
2.1 PEAS Specification	6
2.2 Concept of Rationality	6
3 Environment Properties and Agent Structures	7
3.1 Classification of Environments	7
3.2 Four Basic Agent Architectures	7
3.2.1 1. Simple Reflex Agent	7
3.2.2 2. Model-Based Reflex Agent	7
3.2.3 3. Goal-Based Agent	7
3.2.4 4. Utility-Based Agent	7
Module 2: Problem-Solving Agents & Problem Formulation	10
1 Problem-Solving Agents & Problem Formulation	10
1.1 State Space and Search Trees	10
1.2 Evaluation Criteria for Search Algorithms	10
2 Uninformed Search Strategies	11
2.1 1. Breadth-First Search (BFS)	11
2.2 2. Depth-First Search (DFS)	11
2.3 3. Depth-Limited Search (DLS)	11
2.4 4. Iterative Deepening DFS (IDDFS)	12
2.5 5. Bidirectional Search	12
3 Comparison of Search Strategies	12
Module 3: Informed (Heuristic) Search Strategies	15
1 Informed (Heuristic) Search Strategies	15
1.1 1. Greedy Best-First Search	15
1.2 2. A* Search Algorithm	15
1.3 Conditions for A* Optimality	15
1.4 On-line Search Agents and Unknown Environments	16
2 Constraint Satisfaction Problems (CSPs)	16
2.1 Backtracking Search for CSPs	17
2.2 Constraint Propagation & AC-3 Algorithm	17
2.3 Local Search for CSPs (Min-Conflicts Algorithm)	17

Module 4: Games as Search Problems	21
1 Games as Search Problems	21
1.1 Zero-Sum Property	21
2 The Minimax Algorithm	21
2.1 Properties of Minimax	22
3 Alpha-Beta Pruning	22
3.1 Effectiveness of Alpha-Beta Pruning	22
4 Imperfect Real-Time Decisions	23
Module 5: Knowledge-Based Agents & Wumpus World	26
1 Knowledge-Based Agents & Wumpus World	26
1.1 The Wumpus World Environment	26
2 Propositional Logic	27
2.1 Logical Equivalences	27
3 Inference & Resolution in Propositional Logic	27
3.1 Resolution Rule	27
3.2 Conjunctive Normal Form (CNF) Conversion Steps	27
Module 6: Syntax and Semantics of First-Order Logic	30
1 Syntax and Semantics of First-Order Logic	30
1.1 Quantifiers in First-Order Logic	30
2 Unification & First-Order Inference	30
2.1 Example of Unification	31

Module 1

Overview and Foundations of AI

1. Overview and Foundations of AI

Artificial Intelligence (AI) is the study and design of intelligent agents capable of perceiving their environment, reasoning, learning, and taking actions to achieve specific goals.

Definition of Artificial Intelligence

Artificial Intelligence (AI) is a branch of computer science concerned with building software and systems that perform tasks requiring human-like cognitive abilities, such as learning, reasoning, problem-solving, perception, and decision-making.

1.1 The Four Definitions of AI

AI definitions are historically categorized along two dimensions: **Human Performance vs. Rationality** and **Thinking vs. Acting**.

Dimension	Thinking Focus	Acting Focus
Human-Centric	Thinking Humanly: Cognitive modeling; validating AI models against human brain activity.	Acting Humanly: The Turing Test approach; capability to fool an interrogator.
Rationality-Centric	Thinking Rationally: "Laws of Thought" approach; syllogisms and formal logic.	Acting Rationally: Rational Agent approach; acting to achieve the best expected outcome.

1.2 Foundations of AI

AI is an interdisciplinary field built on insights from multiple disciplines:

- **Philosophy:** Formal logic, theories of mind, epistemology, and nature of knowledge.
- **Mathematics:** Probability, formal logic, statistics, linear algebra, graph theory, and computability theory.
- **Economics:** Decision theory, game theory, utility theory, and operations research.
- **Neuroscience:** Biological neural networks, brain structure, processing mechanisms.
- **Psychology / Cognitive Science:** Human perception, memory models, cognitive architectures.
- **Control Theory & Cybernetics:** Self-regulating feedback systems, optimal control.

1.3 History & Evolution of AI

- **1943–1955 (Gestation):** McCulloch & Pitts artificial neuron model; Turing’s 1950 paper “Computing Machinery and Intelligence”.
- **1956 (Birth of AI):** Dartmouth Conference organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon; coining of the term “Artificial Intelligence”.
- **1966–1974 (Early Enthusiasm):** Logic Theorist, ELIZA, Microworlds (BLOCKSWORLD).
- **1974–1980 (First AI Winter):** Realization of computational complexity limits (NP-completeness).
- **1980–1987 (Expert Systems Era):** MYCIN, XCON (R1), knowledge-based rule systems.
- **1987–1993 (Second AI Winter):** Collapse of specialized LISP machine hardware market.
- **1993–Present (Modern Era):** Probabilistic reasoning, Big Data, Deep Learning, and Large Language Models (LLMs).

1.4 The Turing Test

Proposed by Alan Turing in 1950 (“Imitation Game”), it evaluates whether a computer can demonstrate human-equivalent intelligence.

Capabilities Required for the Turing Test

To pass the Total Turing Test, an AI system requires:

1. **Natural Language Processing (NLP):** To communicate effectively in human language.
2. **Knowledge Representation:** To store information and concepts.
3. **Automated Reasoning:** To use stored knowledge to answer questions and draw conclusions.
4. **Machine Learning:** To adapt to new circumstances and detect patterns.
5. **Computer Vision:** To perceive objects and physical inputs (Total Turing Test).
6. **Robotics:** To manipulate objects and move in physical space (Total Turing Test).

2. Intelligent Agents and Environments

Definition of an Agent

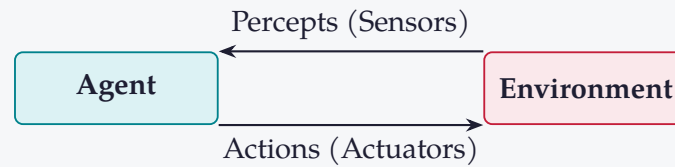
An **Agent** is anything that perceives its environment through **sensors** and acts upon that environment through **actuators**. The mapping from percept sequences to actions is called the **Agent Function**.

Mathematical Formulation of an Agent

$$f : \mathcal{P}^* \rightarrow \mathcal{A} \quad (1)$$

where $\mathcal{P}^*$ is the sequence of all past percepts, and $\mathcal{A}$ is the set of actions available to the agent.

Agent-Environment Interaction Loop



2.1 PEAS Specification

To design an intelligent agent, we must explicitly define its **PEAS** properties:

- **P — Performance Measure:** Objective criteria evaluating the success of the agent's behavior.
- **E — Environment:** External surroundings and domain in which the agent operates.
- **A — Actuators:** Mechanisms through which the agent performs actions.
- **S — Sensors:** Instruments through which the agent receives inputs/percepts.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Automated Taxi Driver	Safe, fast, legal, comfortable, max profit	Roads, traffic, pedestrians, weather	Steering, accelerator, brake, horn	Cameras, LiDAR, GPS, speedometer
Medical Diagnosis	Healthy patient, min cost, no lawsuits	Patient, hospital staff, laboratory	Screen display (reports, prescriptions)	Keyboard (symptoms, lab test results)
Vacuum Cleaner	Clean floor, min time, min electricity	Rooms, dirt, obstacles, carpet	Wheels, vacuum motor, brushes	Bump sensor, dirt detection sensor

2.2 Concept of Rationality

A **Rational Agent** is one that operates to choose actions that maximize its expected performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent possesses.

Factors Determining Rationality

Rationality depends on 4 parameters:

1. The **Performance Measure** defining the success criterion.
2. The agent's **Prior Knowledge** of the environment.
3. The **Actions** that the agent can perform.
4. The agent's **Percept Sequence** up to the present moment.

3. Environment Properties and Agent Structures

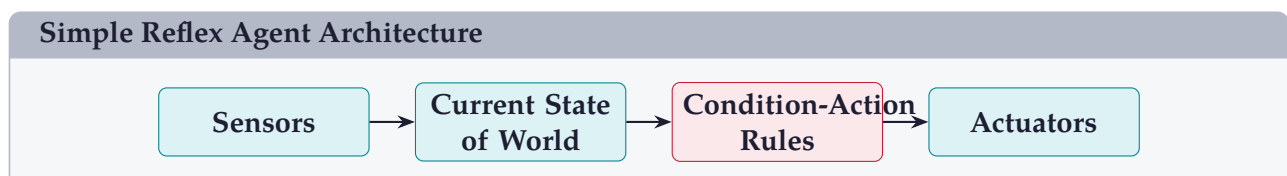
3.1 Classification of Environments

- **Fully Observable vs. Partially Observable:** An environment is fully observable if sensors give complete state access at any time (e.g., Chess vs. Poker / Medical Diagnosis).
- **Single-Agent vs. Multi-Agent:** Single-agent operates alone (Crossword); Multi-agent involves other agents (Chess, Taxi driving). Multi-agent can be competitive or cooperative.
- **Deterministic vs. Stochastic:** Deterministic if next state is completely determined by current state and action (Chess). Stochastic if uncertainty exists (Taxi driving, Ludo).
- **Episodic vs. Sequential:** Episodic means current decision does not affect future decisions (Image classification). Sequential means current actions affect future decisions (Chess).
- **Static vs. Dynamic:** Static environment does not change while agent is deliberating (Crossword). Dynamic environment changes continuously (Taxi driver). Semi-dynamic if environment doesn't change but agent score decreases with time (Timed Chess).
- **Discrete vs. Continuous:** Discrete has finite, distinct states and actions (Chess). Continuous has continuous parameters like speed, angle, temperature (Taxi driver).
- **Known vs. Unknown:** Known means environment laws/physics are given to the agent (Solitaire). Unknown means laws must be learned (New video game).

3.2 Four Basic Agent Architectures

3.2.1 1. Simple Reflex Agent

Selects actions based **only on the current percept**, ignoring percept history. Uses Condition-Action Rules ('IF condition THEN action').



3.2.2 2. Model-Based Reflex Agent

Maintains an internal **state** tracking aspects of the environment not currently visible. Requires knowledge of how the world evolves and how agent actions affect the world.

3.2.3 3. Goal-Based Agent

Combines state information with **goal information** to choose actions that achieve the desired outcome. Uses search and planning.

3.2.4 4. Utility-Based Agent

Uses a **utility function** mapping a state onto a real number to measure happiness/desirability of states. Allows tradeoff choices under conflicting goals.

Quick Revision — Module 1

Introduction to AI & Intelligent Agents | OE-EC804A

Key Concepts and Metrics at a Glance

Agent Function	$f : \mathcal{P}^* \rightarrow \mathcal{A}$ mapping percept histories to actions.
PEAS Specification	Performance Measure, Environment, Actuators, Sensors.
Rationality	Maximizes expected performance based on percept sequence & prior knowledge.
Turing Test	6 core capabilities: NLP, Knowledge Rep, Reasoning, ML, Vision, Robotics.
Environment Dimensions	7 attributes: Observable, Multi-agent, Deterministic, Sequential, Static, Discrete, Known.

One-Line Definitions

- **Rational Agent:** An agent that acts to achieve the best expected performance outcome.
- **PEAS:** Formal framework defining an agent's task environment.
- **Simple Reflex Agent:** Operates strictly on current percept using Condition-Action rules.
- **Model-Based Agent:** Maintains internal state to handle partial observability.
- **Utility-Based Agent:** Evaluates action choices using a quantitative utility function.

Frequently Asked / Viva Questions

Q1. Define Artificial Intelligence. List the four perspectives of AI definitions.

AI is the field of computer science creating systems capable of human-like intelligence. The 4 perspectives are: Thinking Humanly, Acting Humanly (Turing Test), Thinking Rationally (Logic), and Acting Rationally (Rational Agents).

Q2. What is the Turing Test? What capabilities are required to pass it?

Proposed by Alan Turing in 1950, it tests if a machine can imitate human conversation convincingly. Required capabilities: NLP, Knowledge Representation, Automated Reasoning, Machine Learning, Computer Vision, and Robotics.

Q3. What is an Agent and Agent Function?

An agent perceives environment via sensors and acts via actuators. Agent function $f : \mathcal{P}^* \rightarrow \mathcal{A}$ maps percept histories to actions.

Q4. Explain the PEAS framework with an example.

Frequently Asked / Viva Questions (contd.)

PEAS stands for Performance, Environment, Actuators, Sensors. For automated taxi: P=Safety/Speed, E=Roads/Traffic, A=Steering/Brake, S=Cameras/Radar.

Q5. Distinguish between Rationality and Omniscience.

Rationality maximizes expected performance based on available percepts. Omniscience knows actual outcome in advance. Rationality is feasible; omniscience is impossible in uncertain environments.

Q6. Differentiate between Deterministic and Stochastic environments.

Deterministic: Next state is completely fixed by current state and action.
Stochastic: Next state involves probability and randomness.

Q7. What is a Simple Reflex Agent?

An agent that acts based strictly on the current percept using IF-THEN condition-action rules without memory.

Q8. Why does a Model-Based Reflex Agent need internal state?

To handle partially observable environments by tracking unobserved aspects of the world over time.

Q9. Compare Goal-Based vs Utility-Based Agents.

Goal-based agents focus on reaching a target state (binary success). Utility-based agents evaluate degree of desirability (continuous score/utility) to make optimal tradeoffs.

Q10. Explain Episodic vs Sequential environments.

Episodic: Each episode is independent; action choice does not impact future states. Sequential: Actions impact all future states (e.g. Chess).

Module 2

Problem-Solving Agents & Problem Formulation

1. Problem-Solving Agents & Problem Formulation

A **Problem-Solving Agent** is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable states.

Definition of a Problem

A **Problem** is formally defined by 5 components:

1. **Initial State:** $s_0 \in S$, the state where the agent begins.
2. **Actions:** $Actions(s)$, the set of valid actions available in state s .
3. **Transition Model:** $Result(s, a)$, returns the outcome state resulting from taking action a in state s .
4. **Goal Test:** Function determining whether a given state is a goal state.
5. **Path Cost:** $c(s, a, s')$, numeric cost assigned to taking step a from s to s' .

1.1 State Space and Search Trees

- **State Space:** The set of all reachable states from the initial state via any sequence of actions.
- **Search Space / Tree:** A tree generated by the search algorithm where the root is the initial state and child nodes correspond to actions.
- **Branching Factor (b):** Maximum number of successors (children) of any node.
- **Depth (d):** Depth of the shallowest goal node.
- **Maximum Depth (m):** Maximum length of any path in the state space.

1.2 Evaluation Criteria for Search Algorithms

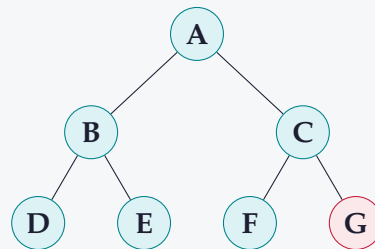
Search algorithms are evaluated based on four fundamental metrics:

1. **Completeness:** Is the algorithm guaranteed to find a solution if one exists?
2. **Time Complexity:** How long (how many nodes generated) does it take to find a solution?
3. **Space Complexity:** How much memory is needed to perform the search?
4. **Optimality:** Does the strategy find the lowest path cost solution?

2. Uninformed Search Strategies

Uninformed search (blind search) strategies have no additional information about states beyond that provided in the problem definition.

Sample Search Tree Topology



2.1 1. Breadth-First Search (BFS)

- **Mechanism:** Expands the root node first, then expands all successors of the root, then their successors. Implemented using a **FIFO (Queue)** frontier.
- **Completeness:** Complete (if branching factor b is finite).
- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b^d)$ (all nodes kept in memory).
- **Optimality:** Optimal if step costs are uniform (equal to 1).

2.2 2. Depth-First Search (DFS)

- **Mechanism:** Expands the deepest node in the current frontier. Implemented using a **LIFO (Stack)** frontier.
- **Completeness:** Incomplete in infinite state spaces or graphs with loops.
- **Time Complexity:** $O(b^m)$
- **Space Complexity:** $O(b \cdot m)$ (linear memory overhead!).
- **Optimality:** Non-optimal (may find a deeper solution before a shallow one).

2.3 3. Depth-Limited Search (DLS)

- **Mechanism:** DFS with a pre-determined depth limit l . Nodes at depth l are treated as if they have no successors.
- **Completeness:** Incomplete if $l < d$.
- **Time Complexity:** $O(b^l)$
- **Space Complexity:** $O(b \cdot l)$
- **Optimality:** Non-optimal if $l > d$.

2.4 4. Iterative Deepening DFS (IDDFS)

- **Mechanism:** Combines the benefits of BFS (completeness/optimality) and DFS (linear memory). It calls DLS repeatedly with increasing depth limits $l = 0, 1, 2, \dots$
- **Completeness:** Complete (if b is finite).
- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b \cdot d)$ (Linear space!).
- **Optimality:** Optimal if step costs are equal.

2.5 5. Bidirectional Search

- **Mechanism:** Runs two simultaneous searches: one forward from initial state and one backward from goal state. Stops when frontiers intersect.
- **Completeness:** Complete if b is finite and both searches are BFS.
- **Time Complexity:** $O(b^{d/2})$
- **Space Complexity:** $O(b^{d/2})$
- **Optimality:** Optimal if step costs are uniform.

3. Comparison of Search Strategies

Criterion	BFS	DFS	IDDFS	Bidirectional
Completeness	Yes (if $b < \infty$)	No	Yes (if $b < \infty$)	Yes (if $b < \infty$)
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^d)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b \cdot m)$	$O(b \cdot d)$	$O(b^{d/2})$
Optimality	Yes (cost = 1)	No	Yes (cost = 1)	Yes (cost = 1)
Frontier Data Struct	FIFO Queue	LIFO Stack	LIFO Stack (Iterative)	Two FIFO Queues

Quick Revision — Module 2

Solving Problems by Searching (Uninformed Search) | OE-EC804A

Key Concepts and Metrics at a Glance

BFS Complexity	Time: $O(b^d)$, Space: $O(b^d)$ (FIFO Queue, Complete, Optimal for step cost = 1)
DFS Complexity	Time: $O(b^m)$, Space: $O(b \cdot m)$ (LIFO Stack, Incomplete, Non-optimal)
IDDFS Complexity	Time: $O(b^d)$, Space: $O(b \cdot d)$ (Linear memory, Complete, Optimal)
Bidirectional Search	Time: $O(b^{d/2})$, Space: $O(b^{d/2})$ (Searches forward & backward)
Problem Components	Initial state s_0 , Actions $Actions(s)$, Transition model $Result(s, a)$, Goal test, Path cost.

One-Line Definitions

- **State Space:** Set of all valid states reachable from initial state.
- **Frontier:** Set of leaf nodes generated but not yet expanded.
- **Branching Factor (b):** Maximum number of successors for any node.
- **IDDFS:** Strategy running DLS with increasing limits $l = 0, 1, 2, \dots$ combining BFS optimality with DFS space.

Frequently Asked / Viva Questions

Q1. What are the 5 components of a formal problem definition?

Initial state, Actions, Transition model, Goal test, and Path cost.

Q2. Compare BFS and DFS in terms of space complexity.

BFS requires $O(b^d)$ space (exponential), whereas DFS requires only $O(b \cdot m)$ space (linear).

Q3. Why is IDDFS preferred over BFS for large state spaces?

IDDFS is complete and optimal like BFS, but uses linear space $O(b \cdot d)$ instead of exponential space $O(b^d)$.

Q4. Why does node regeneration in IDDFS not incur high overhead?

Most nodes in a search tree are at the bottom level (d). Nodes at level d are generated 1 time, level $d - 1$ twice, etc., so total nodes generated $\approx \frac{b}{b-1}b^d$, which is asymptotically $O(b^d)$.

Q5. What is the frontier in search algorithms?

Frequently Asked / Viva Questions (contd.)

The data structure (queue/stack) containing generated nodes that have not yet been expanded.

Q6. When is BFS optimal?

BFS is optimal when all step costs are equal (uniform path cost).

Q7. Explain the time and space complexity of Bidirectional Search.

Time and space complexities are both $O(b^{d/2})$ because two searches meet in the middle.

Q8. What is Depth-Limited Search and why is it used?

DFS restricted to depth limit l . It prevents DFS from getting stuck in infinite paths.

Q9. What is a Graph Search vs Tree Search?

Graph search maintains an 'explored set' to avoid repeating states in looped graphs. Tree search does not.

Q10. Define Completeness and Optimality.

Completeness: Guaranteed to find a solution if one exists. Optimality: Guaranteed to find the lowest-cost solution.

Module 3

Informed (Heuristic) Search Strategies

1. Informed (Heuristic) Search Strategies

Informed search strategies use domain-specific knowledge provided by a **Heuristic Function** $h(n)$ to guide the search towards goal states more efficiently.

Definition of a Heuristic Function

A **Heuristic Function** $h(n)$ estimates the cost of the cheapest path from state n to a goal state. For any goal state g , $h(g) = 0$.

1.1 Greedy Best-First Search

- **Evaluation Function:** $f(n) = h(n)$.
- Expands the node that appears to be closest to the goal.
- **Properties:** Incomplete in looped graphs; non-optimal. Time and Space complexity $O(b^m)$ worst-case.

1.2 A* Search Algorithm

A* combines the cost-so-far $g(n)$ and estimated remaining cost $h(n)$.

A* Evaluation Function

$$f(n) = g(n) + h(n) \quad (2)$$

where:

- $g(n)$: Exact path cost from initial state to node n .
- $h(n)$: Estimated cost from node n to goal state.
- $f(n)$: Estimated total cost of path through n to goal.

1.3 Conditions for A* Optimality

1. **Admissibility (Tree Search Optimality):** A heuristic $h(n)$ is admissible if it **never overestimates** the true cost to reach the goal:

$$h(n) \leq h^*(n) \quad \forall n \quad (3)$$

where $h^*(n)$ is the true minimum cost from n to goal.

2. **Monotonicity / Consistency (Graph Search Optimality):** A heuristic $h(n)$ is consistent if for every node n and successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n') \quad (4)$$

This satisfies the triangle inequality and guarantees $f(n)$ is non-decreasing along any path.

1.4 On-line Search Agents and Unknown Environments

Unlike offline search algorithms (which compute a complete solution path before taking their first action), an **On-line Search Agent** interleaves computation and action: it acts, observes the new state, and decides its next move.

On-line Search Agent

An **On-line Search Agent** operates in unknown or dynamic environments by exploring state space physically. It computes an action, executes it in the environment, senses the resulting state s' , and updates its internal map.

- **Offline vs. On-line Search:** Offline agents compute the full path in a simulator before moving. On-line agents move in the physical world to discover state transitions and costs.
- **Competitive Ratio:** Evaluates on-line search performance by comparing actual path cost incurred to the optimal path cost if the environment were known in advance.
- **LRTA* (Learning Real-Time A*):** An on-line search algorithm that updates heuristic estimates $H(s)$ of visited states based on neighbor cost:

$$H(s) \leftarrow \min_{a \in \text{Actions}(s)} [c(s, a, s') + H(s')] \quad (5)$$

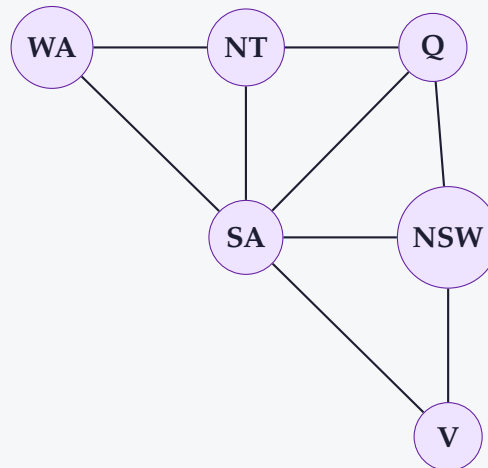
2. Constraint Satisfaction Problems (CSPs)

Definition of a CSP

A **Constraint Satisfaction Problem (CSP)** is defined by a triple (X, D, C) :

- $X = \{X_1, X_2, \dots, X_n\}$: Set of variables.
- $D = \{D_1, D_2, \dots, D_n\}$: Set of domains (possible values for each variable).
- $C = \{C_1, C_2, \dots, C_m\}$: Set of constraints specifying allowable value combinations.

Constraint Graph Representation (Map Coloring Example)



2.1 Backtracking Search for CSPs

Backtracking search is a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left.

Heuristics for CSP Backtracking

1. **Minimum Remaining Values (MRV):** Choose variable with fewest legal values remaining ("fail-first" heuristic).
2. **Degree Heuristic:** Choose variable involved in largest number of constraints on unassigned variables.
3. **Least Constraining Value (LCV):** Choose value that rules out fewest choices for neighboring variables.

2.2 Constraint Propagation & AC-3 Algorithm

Constraint propagation enforces local consistency to prune search spaces early.

- **Forward Checking:** When variable X is assigned, remove inconsistent values from domains of unassigned neighbors.
- **Arc Consistency (AC-3):** Arc $X_i \rightarrow X_j$ is consistent if for every value $x \in D_i$, there exists an allowed value $y \in D_j$.

2.3 Local Search for CSPs (Min-Conflicts Algorithm)

Unlike backtracking search which operates on partial assignments, local search algorithms for CSPs use a **complete-state formulation**: every variable is initially assigned a value (even if constraints are violated).

Min-Conflicts Heuristic

In each iteration, **Min-Conflicts** randomly selects a conflicted variable X_i and reassigns it the value $v \in D_i$ that minimizes the total number of constraint violations across the CSP.

- **Algorithm Mechanism:** Start with a complete assignment $\rightarrow$ pick a conflicted variable X at random $\rightarrow$ set X to value minimizing constraint violations $\rightarrow$ repeat until solution or step limit.
- **Performance:** Solves N -Queens for $N = 1,000,000$ in seconds; space complexity is $O(n)$ (linear).
- **Escaping Local Minima:** Uses random walks, simulated annealing, or random restarts to escape plateau states.

Quick Revision — Module 3

Informed Search & Constraint Satisfaction Problems | OE-EC804A

Key Concepts and Metrics at a Glance

A* Evaluation Function	$f(n) = g(n) + h(n)$ ($g(n)$ cost-so-far, $h(n)$ estimated remaining cost)
Admissibility Condition	$h(n) \leq h^*(n)$ (Heuristic never overestimates true cost to goal)
Consistency Condition	$h(n) \leq c(n, a, n') + h(n')$ (Satisfies triangle inequality along edges)
CSP Formalization	Triple (X, D, C) of Variables, Domains, and Constraints
Min-Conflicts Heuristic	Local search reassigning conflicted variable value to minimize violations.

One-Line Definitions

- **Admissible Heuristic:** A heuristic that never overestimates the actual cost to reach goal.
- **Consistent Heuristic:** A heuristic obeying triangle inequality along graph edges.
- **Arc Consistency (AC-3):** State where every value in a variable domain has a valid partner in neighbor domains.
- **MRV Heuristic:** Chooses the variable with the fewest remaining legal domain choices.

Frequently Asked / Viva Questions

Q1. Define A* search. What is its evaluation function?

A* evaluates nodes using $f(n) = g(n) + h(n)$, combining cost-so-far $g(n)$ and heuristic estimate $h(n)$.

Q2. What is an Admissible Heuristic?

A heuristic $h(n)$ that never overestimates true cost to goal ($h(n) \leq h^*(n)$).

Q3. Differentiate between Admissibility and Consistency.

Admissibility guarantees optimality for tree search. Consistency ($h(n) \leq c(n, a, n') + h(n')$) guarantees $f(n)$ is non-decreasing, ensuring optimality for graph search without re-opening nodes.

Q4. Define CSP formally.

A triple (X, D, C) where X is set of variables, D set of domains, and C set of constraints.

Q5. Explain the MRV heuristic in CSPs.

Frequently Asked / Viva Questions (contd.)

Minimum Remaining Values heuristic chooses the variable with fewest valid remaining choices, aiming to fail early.

Q6. What is Forward Checking?

In CSP search, whenever variable X is assigned, any inconsistent values are pruned from neighboring unassigned variables.

Q7. What is the AC-3 algorithm?

An algorithm that enforces arc consistency across all variable pairs in a CSP constraint graph.

Q8. Compare Greedy Best-First Search and A* Search.

Greedy uses $f(n) = h(n)$ (fast, non-optimal). A* uses $f(n) = g(n) + h(n)$ (optimal if h is admissible).

Q9. What is Manhattan Distance heuristic in 8-puzzle?

Sum of absolute differences in horizontal and vertical coordinates of tiles from goal positions.

Q10. Why is LCV used for value selection in CSPs?

Least Constraining Value maximizes flexibility for remaining variables, leaving maximal choices open.

Module 4

Games as Search Problems

1. Games as Search Problems

In AI, competitive multi-agent environments are modeled as **Adversarial Search Problems** or **Games**.

Formal Definition of a Game

A 2-player deterministic zero-sum game is defined by:

1. S_0 : Initial state of the game board.
2. $Player(s)$: Defines whose turn it is to move in state s .
3. $Actions(s)$: Set of legal moves available in state s .
4. $Result(s, a)$: Transition model returning state outcome of move a in state s .
5. $Terminal-Test(s)$: True if game is over; false otherwise.
6. $Utility(s, p)$: Final numeric value for game outcome state s for player p .

1.1 Zero-Sum Property

In a **Zero-Sum Game**, the total utility across players is constant. For 2 players (MAX and MIN):

$$Utility_{MAX}(s) + Utility_{MIN}(s) = 0 \quad (6)$$

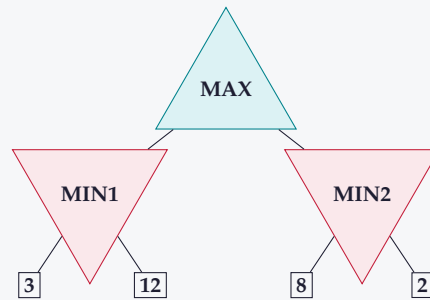
MAX attempts to maximize the utility, while MIN attempts to minimize it.

2. The Minimax Algorithm

Minimax Value Definition

$$Minimax(s) = \begin{cases} Utility(s) & \text{if } Terminal-Test(s) \\ \max_{a \in Actions(s)} Minimax(Result(s, a)) & \text{if } Player(s) = MAX \\ \min_{a \in Actions(s)} Minimax(Result(s, a)) & \text{if } Player(s) = MIN \end{cases} \quad (7)$$

Minimax Game Tree Visualization



2.1 Properties of Minimax

- **Completeness:** Complete if tree is finite.
- **Optimality:** Optimal against an optimal opponent.
- **Time Complexity:** $O(b^m)$ (explores all tree nodes).
- **Space Complexity:** $O(b \cdot m)$ (depth-first traversal).

3. Alpha-Beta Pruning

Alpha-Beta pruning removes branches of the game tree that cannot influence the final decision, reducing search time without altering the optimal choice.

Definition of Alpha (α) and Beta (β)

- **α (Alpha):** Highest-value choice found so far along the path for MAX. (Initial value: $-\infty$).
- **β (Beta):** Lowest-value choice found so far along the path for MIN. (Initial value: $+\infty$).

Alpha-Beta Pruning Condition

Prune remaining children of node n if:

$$\alpha \geq \beta \quad (8)$$

3.1 Effectiveness of Alpha-Beta Pruning

- **Worst-Case Complexity:** $O(b^m)$ (no pruning occurs if worst moves evaluated first).
- **Optimal Move Ordering Complexity:** $O(b^{m/2})$
- With optimal move ordering, Alpha-Beta doubles the solvable search depth!

4. Imperfect Real-Time Decisions

In real games (Chess, Go), searching to terminal leaves is computationally impossible. We replace terminal tests with **Cutoff Tests** and terminal utility with **Evaluation Functions** $Eval(s)$.

Requirements for Evaluation Functions

1. Must rank terminal states in same order as true utility.
2. Computation must be fast.
3. Must correlate strongly with win probabilities (e.g., weighted sum of material values in Chess).

Quick Revision — Module 4

Adversarial Search & Game Playing | OE-EC804A

Key Concepts and Metrics at a Glance

Minimax Decision	$MAX = \max_a \text{Minimax}(\text{Result}(s, a)), \quad MIN = \min_a \text{Minimax}(\text{Result}(s, a))$
α-β Pruning Condition	Prune remaining subtree whenever $\alpha \geq \beta$
Optimal Alpha-Beta Time	$O(b^{m/2})$ (Doubles solvable search depth compared to $O(b^m)$)
Evaluation Function	$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$ (Estimates utility at cutoff)
Zero-Sum Property	$Utility_{MAX}(s) + Utility_{MIN}(s) = 0$ (Constant total utility)

One-Line Definitions

- **Zero-Sum Game:** Game where total utility across all players sums to zero.
- **Alpha (α):** Best (highest) utility value MAX is guaranteed along path.
- **Beta (β):** Best (lowest) utility value MIN is guaranteed along path.
- **Horizon Effect:** Issue where inevitable bad event is pushed beyond search cutoff depth.

Frequently Asked / Viva Questions

Q1. Define a Zero-Sum Game.

A game where one player's gain is exactly equal to the other player's loss ($Util_{MAX} + Util_{MIN} = 0$).

Q2. What is the Minimax algorithm?

A recursive algorithm for calculating optimal moves in 2-player zero-sum games by maximizing MAX's payoff assuming MIN minimizes it.

Q3. Explain Alpha (α) and Beta (β) parameters.

α : Best value for MAX found so far. β : Best value for MIN found so far.

Q4. What is the condition for Alpha-Beta pruning?

Branch is pruned when $\alpha \geq \beta$.

Q5. Does Alpha-Beta pruning change the final outcome of Minimax?

No, it produces the exact same decision as Minimax while pruning irrelevant branches.

Q6. What is the time complexity of Alpha-Beta with perfect move ordering?

Frequently Asked / Viva Questions (contd.)

$O(b^{m/2})$, which doubles the effective search depth compared to standard Minimax ($O(b^m)$).

Q7. Why are evaluation functions needed in game playing?

Because full game trees are too deep to search to terminal states within time limits.

Q8. What is the Horizon Effect?

When search depth limit conceals an unavoidable penalty by delaying it past the cutoff.

Q9. Compare Minimax vs Alpha-Beta Pruning.

Minimax evaluates all $O(b^m)$ nodes; Alpha-Beta prunes provably suboptimal branches reducing nodes to $O(b^{m/2})$.

Q10. What is Quiescence Search?

Search expanded past cutoff depth in non-quiet (unstable) states to prevent misleading evaluation scores.

Module 5

Knowledge-Based Agents & Wumpus World

1. Knowledge-Based Agents & Wumpus World

Definition of a Knowledge-Based Agent

A **Knowledge-Based Agent** maintains an internal **Knowledge Base (KB)** expressed in a formal representation language. It operates by maintaining sentences in its KB, inserting new percepts via **TELL**, and deriving actions via **ASK**.

Core Operations of KB Agent

- **TELL**(KB, sentence): Adds new knowledge or percept to KB.
- **ASK**(KB, query): Queries KB to infer whether query logically follows from KB.

1.1 The Wumpus World Environment

The **Wumpus World** is a benchmark domain for logical reasoning.

- 4×4 grid of rooms containing a Wumpus, Pits, and Gold.
- **Percepts:** [*Stench*, *Breeze*, *Glitter*, *Bump*, *Scream*].
- **Rules:** Rooms adjacent to Wumpus have *Stench*. Rooms adjacent to Pits have *Breeze*. Room containing Gold has *Glitter*.

Wumpus World Grid Environment (4×4)

Stench	Safe	Breeze	PIT
WUMPUS	GOLD	PIT	Breeze
Stench	Safe	Breeze	Safe
Start (1,1) Safe	Breeze	PIT	Breeze

2. Propositional Logic

Propositional logic represents facts using proposition symbols (P, Q, R).

Logical Entailment

Sentence α **entails** sentence β ($\alpha \models \beta$) if and only if in every model in which α is true, β is also true.

2.1 Logical Equivalences

Equivalence Law	Formula
Implication Elimination	$\alpha \implies \beta \equiv \neg\alpha \vee \beta$
Biconditional Elimination	$\alpha \iff \beta \equiv (\alpha \implies \beta) \wedge (\beta \implies \alpha)$
De Morgan's Laws	$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$; $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
Distributivity	$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$

3. Inference & Resolution in Propositional Logic

3.1 Resolution Rule

Unit Resolution rule states that:

$$\frac{l_1 \vee l_2 \vee \dots \vee l_k, \quad \neg l_i}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k} \quad (9)$$

Resolution Refutation proves $KB \models \alpha$ by showing that $KB \wedge \neg\alpha$ is **unsatisfiable**.

3.2 Conjunctive Normal Form (CNF) Conversion Steps

1. Eliminate $\iff$ replacing $\alpha \iff \beta$ with $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$.
2. Eliminate $\implies$ replacing $\alpha \implies \beta$ with $\neg\alpha \vee \beta$.
3. Move $\neg$ inwards using De Morgan's laws ($\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$).
4. Distribute $\vee$ over $\wedge$ ($\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$).

Quick Revision — Module 5

Knowledge Representation & Logical Agents | OE-EC804A

Key Concepts and Metrics at a Glance

Logical Entailment	$KB \models \alpha \iff M(KB) \subseteq M(\alpha) \quad (\alpha \text{ true in all KB models})$
Proof by Contradiction	$KB \models \alpha \iff (KB \wedge \neg\alpha) \text{ is unsatisfiable}$
Implication Elimination	$\alpha \implies \beta \equiv \neg\alpha \vee \beta$
Resolution Rule	$\frac{\alpha \vee \beta, \neg\beta \vee \gamma}{\alpha \vee \gamma} \quad (\text{Derives empty clause on contradiction})$
CNF Structure	Conjunction of disjunctions of literals (AND of OR clauses)

One-Line Definitions

- **Knowledge Base (KB):** A set of sentences expressed in a formal representation language.
- **Model:** A mathematical world assigning truth values (True/False) to every proposition.
- **CNF:** A conjunction of disjunctions of literals (AND of ORs).
- **Sound Inference:** Derives only entailed sentences (no false conclusions).
- **Complete Inference:** Can derive any sentence that is entailed.

Frequently Asked / Viva Questions

Q1. Define a Knowledge-Based Agent.

An agent that reasons over a formal Knowledge Base (KB) using TELL and ASK operations to select actions.

Q2. What is Entailment ($\models$)?

$KB \models \alpha$ means α is true in all models where KB is true.

Q3. What is the difference between Syntax and Semantics?

Syntax defines allowable sentence structures. Semantics defines sentence meaning and truth values in models.

Q4. State De Morgan's Laws.

$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ and $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$.

Q5. What is Conjunctive Normal Form (CNF)?

A logical sentence format structured as an AND of clauses, where each clause is an OR of literals.

Q6. How does Resolution Refutation work?

Frequently Asked / Viva Questions (contd.)

To prove $KB \models \alpha$, add $\neg\alpha$ to KB and apply resolution rules until an empty clause (contradiction) is derived.

Q7. Differentiate Soundness and Completeness.

Soundness: Everything derived is true ($KB \vdash \alpha \implies KB \models \alpha$). Completeness: Every true fact can be derived ($KB \models \alpha \implies KB \vdash \alpha$).

Q8. What is Forward Chaining?

An inference algorithm for Horn clauses that starts from known facts and applies rules to infer new facts until query is proved.

Q9. What is Backward Chaining?

An inference algorithm that starts from query goal and works backward through rules to find supporting facts.

Q10. What is a Horn Clause?

A disjunction of literals containing at most one positive literal.

Module 6

Syntax and Semantics of First-Order Logic

1. Syntax and Semantics of First-Order Logic

First-Order Logic (FOL) extends propositional logic by representing objects, relations, and functions explicitly.

Elements of First-Order Logic

- **Objects:** Concrete entities in the world (e.g., *John*, *5*, *CGEC*, *Wumpus*).
- **Relations / Predicates:** Properties or connections between objects (e.g., *Brother(John, Richard)*, *Greater(5, 3)*).
- **Functions:** Operations mapping objects to another object (e.g., *MotherOf(John) = Mary*).
- **Quantifiers:** $\forall$ (Universal: "for all") and $\exists$ (Existential: "there exists").

1.1 Quantifiers in First-Order Logic

- **Universal Quantifier ($\forall$):** $\forall x P(x)$ means $P(x)$ is true for all objects x in the domain. *Example:* "All students are smart" $\rightarrow \forall x (Student(x) \Rightarrow Smart(x))$.
- **Existential Quantifier ($\exists$):** $\exists x P(x)$ means $P(x)$ is true for at least one object x in the domain. *Example:* "Some student is smart" $\rightarrow \exists x (Student(x) \wedge Smart(x))$.

Quantifier Duality Rules

$$\forall x \neg P(x) \equiv \neg \exists x P(x) \quad ; \quad \neg \forall x P(x) \equiv \exists x \neg P(x) \quad (10)$$

2. Unification & First-Order Inference

Definition of Unification

Unification is an algorithmic process of finding a substitution θ that makes two first-order logic expressions identical.

Unification Function

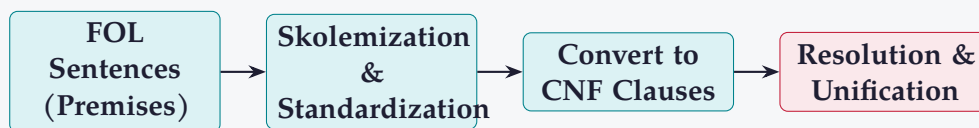
$$Unify(p, q) = \theta \quad \text{such that } SUBST(\theta, p) = SUBST(\theta, q) \quad (11)$$

where θ is the **Most General Unifier (MGU)**.

2.1 Example of Unification

Suppose we want to unify $Knows(John, x)$ and $Knows(John, Jane)$:

$$Unify(Knows(John, x), Knows(John, Jane)) = \{x/Jane\} \quad (12)$$

First-Order Logic Resolution Pipeline

Quick Revision — Module 6

First-Order Logic (FOL) | OE-EC804A

Key Concepts and Metrics at a Glance

Universal Quantifier	$\forall x(P(x) \implies Q(x))$ (True for all objects in domain)
Existential Quantifier	$\exists x(P(x) \wedge Q(x))$ (True for at least one object in domain)
Quantifier Duality	$\neg \forall x P(x) \equiv \exists x \neg P(x)$; $\neg \exists x P(x) \equiv \forall x \neg P(x)$
Unification Function	$SUBST(\theta, p) = SUBST(\theta, q)$ (θ is Most General Unifier)
Skolemization	Replaces existential variable $\exists x$ with Skolem constant/function

One-Line Definitions

- **Predicate:** A logical function returning True/False based on object relations.
- **Unification:** Algorithm that computes variable substitutions to make two expressions match.
- **Skolemization:** Process of replacing existential variables with Skolem constants/functions.
- **MGU (Most General Unifier):** The simplest substitution that unifies two logic expressions.

Frequently Asked / Viva Questions

Q1. Compare Propositional Logic and First-Order Logic.

Propositional logic assumes world contains atomic facts. FOL assumes world contains objects, relations, functions, and quantifiers.

Q2. Explain the use of Universal Quantifier ($\forall$).

Expresses that a property holds for all objects in the domain. Typically used with implication ($\implies$).

Q3. Explain the use of Existential Quantifier ($\exists$).

Expresses that a property holds for at least one object in domain. Typically used with conjunction ($\wedge$).

Q4. What is Unification in FOL?

Finding a substitution set θ that makes two predicate expressions identical.

Q5. What is Skolemization?

Eliminating existential quantifiers $\exists x$ by substituting x with a Skolem constant or function depending on outer universal variables.

Frequently Asked / Viva Questions (contd.)**Q6. State the Quantifier Duality rule.**

$\neg\forall xP(x) \equiv \exists x\neg P(x)$ and $\neg\exists xP(x) \equiv \forall x\neg P(x)$.

Q7. Why is $\wedge$ usually incorrect with $\forall$?

$\forall x(Student(x) \wedge Smart(x))$ asserts that *everything in the universe* is a student and is smart! $\Rightarrow$ should be used instead.

Q8. What is Generalized Modus Ponens?

An inference rule in FOL that combines Modus Ponens with Unification to infer conclusions directly from KB.

Q9. What is a Most General Unifier (MGU)?

A unifier that places the fewest restrictions on variables.

Q10. What is Ground Resolution?

Resolution applied to ground (variable-free) sentences.